Introduction

Thank you for your application. We would like to invite you to complete a coding interview as the next step in the process.

Once you have completed the task, please submit your work via GitHub. The assessment consists of **5 questions**:

- Section A: 3 questions
- Section B: 1 question
- Section C: 1 question

Please create **separate folders for each question** in your repository.

Regarding the **IT Support Coding Interview submission**, we would like to update the submission method.

Instead of replying via email with your GitHub link, please submit your GitHub repository through the Microsoft Form below:

- https://forms.office.com/r/FRWHJdhKQ4

If your repository is set to **private**, please make sure to grant access to the following email addresses before submitting:

- qwer-1234-q
- yulong-rgb

This will help us review your submission more efficiently.

The deadline for submission is **9:00 am, 17 April 2026 (Sydney time)**.

As this role is primarily focused on IT support, the coding tasks are designed to assess your problem-solving approach rather than advanced programming skills. If you find any of the questions challenging, please do not worry — simply complete what you can. You are not expected to finish everything.

Thank you for your understanding, and please let us know if you have any questions.

We look forward to your submission.

Solum

Section A Data analysis

This dataset presents a high-granularity structured reconstruction of BMW Group global deliveries, premium segment share, EV penetration, and macroeconomic indicators from 2018–2025.
Designed for time-series forecasting, EV adoption modelling, and premium automotive market research.

About the dataset columns:

1. Year: Calendar year of vehicle sales between 2018 and 2025.
2. Month: Month of Sale (1-12).
3. Region: Geographic market where the vehicles were sold.
   - Europe
   - China
   - USA
   - RestOfWorld
4. Model: BMW model variant.
5. Units_Sold: Number of vehicles delivered in that region, month, and model.
6. Avg_Price_EUR: Estimated average transaction price per vehicle in Euros.
7. Revenue_EUR: Total estimated revenue.
8. BEV_Share: Battery Electric Vehicle penetration percentage (0–1 scale).
9. Premium_Share: BMW's estimated share within the premium automotive segment in that region.
10. GDP_Growth: Regional GDP growth rate (%).
11. Fuel_Price_Index: Relative fuel cost index.

Solum

Question A

How does **BEV_Share growth over time (2018–2025)** correlate with **Units_Sold and Revenue_EUR** across different regions, and which region shows the strongest transition toward electrification?

Question B

Which **models demonstrate the highest price elasticity**, based on changes in **Avg_Price_EUR vs Units_Sold**, and how does this vary across economic conditions (GDP_Growth levels)?

Question C

Can we identify **seasonal patterns (Month-level trends)** in **Revenue_EUR and Units_Sold**, and do these patterns interact differently with **regional economic indicators (GDP_Growth, Fuel_Price_Index)**?

Solum

Section B Algorithm

🌳 Question D: Tree, TREE!!!

You are given a tree with **n nodes**.

For any chosen root **r**, we define a value called the **cuteness** of the tree as follows:

- Consider **all possible sets of k distinct nodes**.
- For each set, compute their **Lowest Common Ancestor (LCA)** when the tree is rooted at **r**.
- Collect all such LCA results into a set $\mathbf{S_r}$ (only distinct nodes).
- The **cuteness** for root **r** is the size of this set:

$$f(r) = |S_r|$$

💖 Goal: Kawaiiness

Compute the total cuteness over all possible roots:

$$\sum_{r=1}^{n} f(r) = |S_1| + |S_2| + \cdots + |S_r|$$

Solum

📋 Input

- First line: integer **t** — number of test cases
- For each test case:
  - First line: integers **n** and **k**
    - $2 \le k \le n \le 2 \cdot 10^5$
  - Next **n−1** lines: edges of the tree
    - Each line contains **u v**, meaning an edge between nodes u and v

⚠️ The graph is guaranteed to be a tree.

⚠️ Total sum of all **n** across test cases ≤ $2 \cdot 105$

Output

For each test case, output a single integer — the **kawaiiness** of the tree.

Key Idea

For each root:

- Try all subsets of size **k**
- Compute their LCA
- Count how many **distinct nodes** can appear as LCAs

Then sum this over all roots.

Example

Input	Output
4 2 2 1 2 5 3 1 2 1 3 1 4 1 5 6 3 1 2 1 3 2 4 2 5 3 6 10 5 ...	2 9 17 35

Solum

✨ Intuition

- Each root changes the structure of the tree (parent/child relationships).
- That affects which nodes can become LCAs of k chosen nodes.
- Your task is to efficiently compute how many nodes can act as LCAs across all roots.

📝 Note

Let $f(i) = |S_i|$.

For the third example:

- **Root = 1**
  Only nodes **1** and **2** can appear as LCAs.
  - Example: $LCA(4,5,6) = 1$
  - Example: $LCA(2,4,5) = 2$
    So, $f(1) = 2$

- **Root = 2**
  Only nodes **1** and **2** can appear.
  - Example: $LCA(1,3,6) = 1$
  - Example: $LCA(1,4,5) = 2$
    So, $f(2) = 2$

- **Root = 3**
  Nodes **1, 2, 3** can appear.
  - Example: $LCA(2,4,6) = 3$
    So, $f(3) = 3$

- **Root = 4**
  Nodes **1, 2, 4** can appear.
  - Example: $LCA(1,3,5) = 2$
    So, $f(4) = 3$

Solum

- **Root = 5**
  Nodes **1, 2, 5** can appear.
  - Example: $LCA(3,4,6) = 2$
    So, $f(5) = 3$

- **Root = 6**
  Nodes **1, 2, 3, 6** can appear.
  - Example: $LCA(3,4,5) = 2$
    So, $f(6) = 4$

☑ Final Answer

$$f(1) + f(2) + f(3) + f(4) + f(5) + f(6) = 2 + 2 + 3 + 3 + 3 + 4 = 17$$

Solum

Section C Full – Stack Interview Assignment

Question E

CMS Dialysis Facility Data – Mortality Analysis

🎯 Objective

Build a small full-stack application to analyze **Mortality Rate (Facility)** using the provided dataset.

The app should allow users to **filter, compare, and analyze mortality rates** across time and location.

📦 Dataset

https://data.cms.gov/provider-data/docs

You can access the database above.

Solum

🧩 Requirements

1️⃣ Backend (Required)

Build a backend service that:

A. Filtering

Supports filtering by:

- Year
- Month
- State
- ZIP Code (postcode)
- Facility Name (search)

B. Endpoints

1. Summary Endpoint

Returns:

```
{
  "total": number,
  "avgMortality": number,
  "minMortality": number,
  "maxMortality": number,
  "top10Highest": [...],
  "top10Lowest": [...]
}
```

2. Table Endpoint (Paginated)

Returns:

```
{
  "data": [...],
  "page": number,
  "pageSize": number,
  "total": number
}
```

Solum

3. Analysis Endpoint

Returns aggregated data:

```
{
  "monthlyTrend": [...],
  "byState": [...],
  "byZip": [...],
  "distribution": [...]
}
```

C. Logic Requirements

- Parse date → extract **year and month**
- Handle missing or invalid mortality values
- Sort correctly for rankings
- Implement pagination
- Ensure consistent response format

2 Frontend (Required)

Build a frontend with **2 pages**:

Page 1: Summary

Filters

- Year (dropdown)
- Month (dropdown)
- State (dropdown)
- ZIP Code (input)
- Facility Name (search input)

Display

- Total facilities
- Average mortality rate
- Max mortality rate
- Min mortality rate
- Top 10 highest mortality facilities
- Top 10 lowest mortality facilities
- Full data table (paginated)

Solum

Page 2: Analysis

Display visual insights such as:

- Mortality rate trend by month
- Mortality comparison by state
- Mortality comparison by ZIP code
- Distribution (histogram or grouped buckets)
- Facility ranking table

🧠 Evaluation Criteria

We will evaluate:

Backend

- Clean API design
- Correct filtering logic
- Aggregation correctness
- Handling of edge cases (missing data, invalid values)

Frontend

- Clear UI structure
- Proper state management
- Good use of charts/tables
- Responsive to filter changes

Overall

- Code organization
- Readability
- Simplicity and correctness over complexity

Solum

⭐ Bonus (Optional)

- Compare multiple states side-by-side
- Export filtered results (CSV)
- Caching or performance optimization
- Highlight outliers
- Show national vs state averages

⏱ Expected Time

~2–4 hours (no need to over-engineer)

🚀 Submission

Provide:

- Source code (GitHub)
- Instructions to run locally
- (Optional) short explanation of decisions

💡 Notes

- Keep it simple and working
- Focus on **data correctness and usability**
- You do NOT need perfect UI styling

Solum